Project Brief

Product

- **One-line vision:** Repair one known public stack-buffer-overflow vulnerability in the same Linux x86-64 ELF through two AI-assisted routes, and compare the validation evidence in a local browser interface.
- **Target users:** Developers and security analysts at a client software company who hold a vulnerable binary and no original source. The five-student team operates the first demonstration with reviewer assistance.
- **Primary user outcome:** A repeatable workflow that connects binary analysis, two independent repair routes, shared validation, and auditable artifacts a second person can replay.
- **Secondary user:** The reviewer, who confirms the repair location and security contract before any automated repair runs, and judges the evidence afterwards.

Scope

- Must ship (all five are P0):
  - **F1 — Graphical job setup and contract review.** Accept the ELF, defect description, reproducer, valid-input tests, and environment information; record input identity by cryptographic hash; reject wrong architecture, malformed files, and missing inputs before execution; let a reviewer see and confirm the repair location and security contract.
  - **F2 — Direct instruction repair (Route A).** Generate candidate machine-instruction changes from binary-derived analysis and the reviewed contract; check encoding, instruction boundaries, placement, and affected transfers; apply to a separate copy; report `unsupported` with a reason when safe accommodation is impossible.
  - **F3 — Recovered C repair and reintegration (Route B).** Recover C-like code for a local region without original source, keep the unedited output and every manual correction, pass an **unchanged round trip** before any security edit, then compile and reintegrate into a separate copy.
  - **F4 — Shared validation and explicit outcomes.** Both candidates face the same test inputs, expected outcomes, reviewed contract, and analysis bounds on the **actual final patched executables**: structural, compatibility, behavioural, and bounded symbolic checks, with the symbolic domain, assumptions, and unresolved exploration recorded.
  - **F5 — Route comparison and artifact export.** Show both routes side by side including partial and unsuccessful outcomes, with changed/added regions, test counts, verification and compatibility results, elapsed time, manual interventions, and exportable HTML + JSON evidence bound to matching hashes.
- Not in v1:
  - Named real-world CVE repair (next milestone) and the final production application.
  - Arbitrary vulnerability discovery — the initial location and contract are reviewer-confirmed.
  - Broad architecture or binary-format support; the MVP is Linux x86-64 ELF only, and complex or unmodeled ELF features may be reported `unsupported`.
  - General recovery of original source — Route B recovers only what its local repair requires.
  - Unconditional whole-program correctness proof, and any claim of a globally smallest patch.

    - Automatic client deployment or hot patching; export exists for review, not production writeback.
    - Hosted processing of confidential client binaries without explicit authorization.
    - Public SaaS, payments, social features, mobile application, enterprise accounts, multi-tenancy.
- Nice-to-have: none committed. Additional usability work must not displace a required route or a validation obligation.

Constraints and delivery boundaries

- **Two-route scope is a hard requirement for MVP acceptance.** One successful route does not satisfy it, and a deterministic baseline repair cannot stand in for the AI-assisted route.
- **Same input for both routes:** one public binary, one defect, one reviewed contract, one set of test inputs.
- **No terminal use by the operator** during the demonstration once the environment is prepared. Environment installation may still require technical setup.
- **Capacity:** five members × five hours per week = 25 person-hours per week, including learning, review, integration, and demonstration preparation.
- **Timeline:** the proposal and a progress demonstration are due approximately 22 October 2026 (exact time TBD). The progress demonstration may show a working GUI with a partial pipeline and unfinished stages visible; the completed-MVP date follows feasibility gates and is not yet estimated.
- **Evidence honesty:** targets are acceptance requirements, not achieved measurements, and no population-level repair success may be claimed from one sample.

Outcome semantics

Only these labels exist, and they must remain visibly distinct in the UI:

Outcome	Meaning
`success_within_scope`	Every required obligation passed, with matching hashes and a nonempty reviewed scope
`unsupported`	A required capability is outside supported conditions — explain the limitation
`rejected`	A generated candidate failed a required check — show the reason and keep the evidence
`inconclusive`	Available evidence cannot establish a required property — show the unresolved obligation
`error`	A tool, provider, or system failed — show stage and reason, keep completed evidence
`cancelled`	The operator stopped the job — stop work and label retained evidence accordingly

Principles

- Solve the user story before adding polish.
- Prefer boring, maintainable choices; check the project's real dependencies before adding one.
- AI proposes, deterministic checks decide. A model explanation is never proof.
- Keep generated docs short and current.
- Verify user-visible work in the real product surface, and state plainly what was not verified.
- Report a blocker instead of silently dropping a route, a check, or a limitation.

AI Position

- AI is used for: proposing repair candidates (Route A instruction selection/parameterization, Route B constrained security edits) and assisting development of this project.
- AI is not used for: deciding acceptance, choosing its own target locations or interfaces, handling client binaries or confidential derived content by default, or replacing any verification gate.
- Human approval required for: reviewer confirmation of the repair location and security contract before automated runs; explicit client authorization for any external transfer of client material.